Ders 5 Genişletilmiş Notu: Çoklu Programlama, İkili Mod, İşletim Sistemi Görevleri, Koruma ve Sistem Çağrıları

Ders 5 Genişletilmiş Notu: Çoklu Programlama, İkili Mod, İşletim Sistemi Görevleri, Koruma ve Sistem Çağrıları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kaldığımız Yer
- 2. Çoklu Programlama ve Zaman Paylaşımı
- 3. Kesmelerle Sürülen İşletim Sistemi
- 4. İkili Mod: Kullanıcı Modu ve Çekirdek Modu
- 5. Süreç Yönetimi
- 6. Bellek Yönetimi
- 7. Depolama Yönetimi ve Dosya Sistemi
- 8. Depolama Hiyerarşisi ve Veri Tutarlılığı
- 9. G/Ç Alt Sistemi
- 10. Koruma ve Güvenlik
- 11. Çekirdekte Kullanılan Veri Yapıları
- 12. İşletim Sistemi Yapıları: Servisler
- 13. Kullanıcı Arayüzleri: Komut Satırı ve Grafik Arayüz
- 14. Sistem Çağrıları ve API
- 15. Sistem Çağrısının İşlenmesi ve Parametre Aktarma
- 16. Sistem Çağrısı Türleri
- Örnek Sorular
- Dersin Toparlaması

1. Kaldığımız Yer

Önceki derste işletim sisteminin kesmelerle sürülen (interrupt driven) bir yapı olduğu konuşulmuş ve orada durulmuştu. Hangi slaytta kalındığı tam hatırlanmadığı için hoca, garanti olsun diye 1. bölümün 29. slaytına, yani çoklu programlamaya (multiprogramming) geri döndü ve oradan devam etti. Böylece bazı konular ikinci kez dinlenmiş oldu.

2. Çoklu Programlama ve Zaman Paylaşımı

2.1 Kaynakların etkin kullanımı

Hocaya göre bilgisayar sistemindeki her şey bir kaynaktır: işlemci bir kaynaktır, bellek başka bir kaynaktır. Bu kaynakların etkin kullanılması gerekir.

İşlemcide çalıştırdığımız bir sürecin (process) yaşam döngüsü iki tür kısımdan oluşur. Bu döngü zamanlama (scheduling) konusunda tekrar ele alınacak.

- **CPU ani yükü (CPU burst):** İşlemcinin yoğun kullanıldığı kısım.

- **G/Ç ani yükü (I/O burst):** Giriş-çıkışın yoğun olduğu kısım. Süreç burada bir şeyin tamamlanmasını bekler: bir yere bilgi aktarır ya da bir yerden bilgi gelmesini bekler.

G/Ç kısmında işlemci belli açılardan atıl kalır. Çalıştırılabilecek bir işlemci kaynağı vardır, ama o sürecin işlemcide yapacağı bir iş yoktur. Bu boşluğu değerlendirmek için kullanılan mekanizma çoklu programlamadır.

2.2 Çoklu programlama

Çoklu programlamada amaç, bellekte herhangi bir anda tek bir program değil, birden fazla program (örneğin 6 program) bulunmasıdır. Bunlardan biri CPU'yu kullanmak için seçilir. Süreç tek bir döngüden ibaret değildir: CPU yoğun ve G/Ç yoğun kısımlar dönüşümlü gelir. Süreç bir G/Ç başlattığında, okuma ya da yazma tamamlanmadan devam edemez ve beklemesi gerekir. CPU'nun boşuna beklememesi için bellekteki başka bir program çalıştırılır. Bunun için birden fazla programın belleğe alınması ve süreçlerin ayrı ayrı tutulması gerekir.

2.3 Zaman paylaşımı

Çoklu programlamada bir sorun kalır. Bir sürecin CPU yoğun kısmı çok uzunsa, diğer süreçler hiç ilerlemiyormuş gibi görünebilir. Hoca bunu günlük deneyimlerle anlattı: Bir tarayıcıyı ya da bir uygulamayı açmak için tıklarsınız, bekledikçe beklersiniz. Arkada başka bir süreç çalışıyordur ve sizinkine sıra gelmiyordur. Hoca bunun tek senaryo olmadığını da ekledi: Çalıştırdığınız program arkada çok fazla G/Ç yapıyor da olabilir. Belirtileri de tanıdıktır: Fareyi hareket ettirirsiniz ama imleç geriden gelir, klavyede bastığınız tuşlar ekrana geç düşer.

İşlerin daha etkileşimli olmasını, işlemcinin aynı anda birden fazla programı çalıştırıyormuş gibi görünmesini istiyorsak zaman paylaşımı (time sharing) kullanılır. Hoca burada çok işlemciden değil, **tek işlemci ve çok sayıda süreçten** bahsettiğini vurguladı. Çok işlemcili duruma geçince de durumun çok farklı olmadığını, onun da yeri gelince konuşulacağını söyledi.

Zaman paylaşımında işlemcideki süreci değiştirmek için CPU yoğun kısmın bitmesi beklenmez. Belli zaman aralıklarıyla işler arasında parça parça geçilir: önce bu süreçten bir parça, sonra şundan bir parça, sonra ötekinden bir parça. Böylece dairesel (circular) bir yapı kurulur. Bu yapının illa dairesel olması gerekmez, farklı ölçütlere göre de kurulabilir. Bu da zamanlama konusunda işlenecek.

Amaç tepki süresinin (response time) düşmesidir, çünkü büyük ihtimalle daha etkileşimli bir ortamda çalışıyoruz. Yani zaman paylaşımında süreçler arasındaki geçiş, CPU ya da G/Ç yoğun kısımların kendiliğinden bitmesine bırakılmaz, planlı programlı yapılır.

2.4 Takas ve sanal bellek

Zaman paylaşımının bir şartı, bellekte bol miktarda süreç olmasıdır. Fakat bir noktadan sonra süreç oluşturma devam eder. Kullanıcı "bellek doldu, ben süreç oluşturmayayım" demez. Sunucu sistemlerinde de kaynaklarınız dolu diye istek gelmesi durmaz. Çalışan süreç sayısı artınca, daha az kullanılanların ikincil belleğe (secondary storage) atılması, etkileşimi sağlamak için gerekince geri getirilmesi gerekir. Yani süreçleri ikincil bellek ile ana bellek arasında taşıyan ve bunu bir takvime oturtan (çizelgeleyen) bir mekanizma gerekir. Buna takas (swapping in ve swapping out) denir. Bu da zamanlama konusunda daha ayrıntılı işlenecek.

Bellek ne kadar büyük olursa olsun bu sorun bitmez. Dizüstü bilgisayarlarda 8 GB, 16 GB bellek vardır, sunucu sistemlerinde terabayt düzeyinde bellek olabilir. Ama süreç sayısı arttıkça bellek ihtiyacı da artar. Bellek sınırına geldik diye hiçbir şey çalıştırmamak gibi bir şansımız yoktur. Bunun için sanal bellek (virtual memory) kullanılır. Sanal bellek, süreçlerin bir kısmının fiziksel bellek dışında saklanmasını, bir kısmının fiziksel bellekte olmasını sağlar. Böylece bellekte yalnızca kısmen bulunan süreçler de çalıştırılabilir.

2.5 Bellekteki yerleşim

Slaytta çok sembolik bir 512 MB'lık bellek gösteriliyordu. En üstte işletim sistemi, altında işler (job) duruyordu. Hoca işletim sisteminin en üstte olmak zorunda olmadığını, en altta da olabileceğini söyledi. Önemli olan bellekte birden fazla işin bulunmasıdır.

Bir iş tamamlanınca, yani program çalışıp normal biçimde sonlanınca (terminate), bellekten çıkarılması gerekir. Örneğin 2 ve 4 numaralı işler bellekten çıkarıldı. Bellekte iki "delik" oluştu. Sonra 5 numaralı yeni bir iş geldi, ama ne 2'nin ne de 4'ün yerine sığıyor. Bu durumda ne yapılacağı bellek yönetimi konusunda konuşulacak.

3. Kesmelerle Sürülen İşletim Sistemi

Hoca önceki derste söylediğini hatırlattı: Kesme (interrupt) mekanizması olmadan modern bir işletim sisteminden bahsetmek mümkün değildir. Modern olmayanı da vakit kalırsa konuşulacaktı. Kesme, ders boyunca çokça konuşulacak bir mekanizmadır. Örneğin her sistem çağrısında aslında bir kesme mekanizması kullanılır.

> **Hocanın vurgusu:** Kesmenin ne olduğunu ve nasıl çalıştığını anlamak önemlidir.

Kesmeler farklı biçimlerde sınıflandırılabilir. En temel sınıflandırma donanım ve yazılım kesmesi ayrımıdır.

- **Donanım kesmesi (hardware interrupt):** Bir aygıtın ürettiği kesmedir. Bir G/Ç aygıtından, örneğin diskten gelebilir. Klavye gibi bir karakter aygıtından (character device) da gelebilir. Başka bir arayüzden ya da donanımdan da gelebilir.
- **Yazılım kesmesi:** Buna zaman zaman istisna (exception) ya da tuzak (trap) da denir. Kaynakları şunlar olabilir:
  - **Çalışma zamanı hatası:** Burada "yazılım hatası" ile bir şeyin yanlış kullanılması kastedilmez. Çalışma zamanında (runtime) oluşan bir hatadan bahsedilir. En çok verilen örnek sıfıra bölmedir (division by zero).
  - **Servis isteği:** İşletim sisteminden bir servis almak istiyorsanız sistem çağrısı (system call) yaparsınız. Sistem çağrısı yapmak için de aslında bir kesme oluşturursunuz. Bunu doğrudan yapmayız, sistem çağrısını kullanırız. Hatta sistem çağrısını da doğrudan kullanmayız. Bu konu 2. bölüm slaytlarında ayrıntılı işlenecek.
  - **Haberleşme** için de kullanılır.
  - **Denetim mekanizmaları:** Bir işletim sistemindeki süreçlerin birbiriyle iyi geçinmesi beklenir. Bir süreç başka bir süreci taciz etmemeli, onu değiştirememelidir. Süreçler birbirinden korunmalı, kendi bütünlüklerini sağlayabilmelidir. Bir süreç başka bir sürecin belleğini değiştirmeye kalkarsa ya da yetkisinde olmayan bir komutu (instruction) çalıştırmaya kalkarsa bir yazılım kesmesi, yani istisna ya da tuzak oluşmalıdır.

4. İkili Mod: Kullanıcı Modu ve Çekirdek Modu

4.1 Neden iki mod gerekir?

Elimizde işlemci, bellek ve çevre birimlerinden oluşan bir donanım vardır. Bu kaynakları yöneten yazılıma işletim sistemi deriz. İlk derste de konuşulduğu gibi, kimse bu yapının arasından sıyrılıp "ben şuradan şunu alayım" diyememelidir. Bu yüzden donanım üzerinde çalışan yazılımları sınıflayabilmemiz, birbirinden ayırt edebilmemiz gerekir.

İşlemcide toplama, çıkarma gibi herkesin çalıştırabileceği komutlar vardır. Bunlarda sorun yoktur. Ama bazı komutlar yalnızca yetkili tarafından çalıştırılmalıdır:

- zamanlayıcıya (timer) erişip onu programlamak,
- sayfalama (paging) ile ilgili bellek yönetimi komutları,
- bir süreç başlatıldığında ona ayrılan bölümlerin (segment) okunabilir, yazılabilir ya da çalıştırılabilir olacağını belirlemek.

Bunlar da komutlarla yapılır ve bu komutları çalıştıran yetkili taraf işletim sistemidir. O zaman bir komutu çalıştıranın işletim sistemi mi, kullanıcı mı olduğunu anlamamız gerekir.

Bunu yazılımın kendisine soramayız. Yazılıma "Sen işletim sistemi misin?" diye sorarsanız "Evet, işletim sistemiyim" diyebilir. Bu yüzden ayrım yalnızca yazılımla yapılamaz. Yalnızca donanımla da yapılamaz. İki tarafın da desteklediği ortak bir yapıya ihtiyaç vardır.

> **Hocanın vurgusu:** Bir komutu kimin çalıştırdığı yalnız yazılımla anlaşılamaz. İkili mod, donanım ile yazılımın birlikte desteklediği bir yapıdır.

4.2 Mod biti

Bu destek donanım tarafında, işlemcinin doğrudan desteklediği çalışma modlarıyla sağlanır. Temelde iki moddan bahsedilir, bu yüzden buna ikili mod (dual mode) denir:

- **Kullanıcı modu (user mode):** Kullanıcı seviyesi.
- **Çekirdek modu (kernel mode):** Çekirdek seviyesi.

Hangi modda olunduğunu bir **mod biti** gösterir. Bitin 0 ya da 1 olmasına göre çekirdek ya da kullanıcı modunda olunur. Hangi değerin hangi moda karşılık geldiği mimarinin verdiği bir karardır. İşletim sistemi de bu karara uygun çalışmalıdır.

4.3 Modlar arası geçiş

Bu yapı açılışta şöyle kurulur. İşletim sisteminin başlaması için bir önyükleyici (bootloader) vardır, ondan önce de bir başlatıcı (bootstrap) vardır. Bu, donanım üzerinde yazılı olan aygıt yazılımıdır (firmware). Bilgisayar sistemlerinde buna daha çok BIOS denir. BIOS devreye girer ve önyükleyicinin başlatılmasını sağlar. Önyükleyici de işletim sistemini başlatır. Önyükleyici ile işletim sistemi zaten aynı seviyededir. BIOS, önyükleyiciyi başlatırken sistemi çekirdek modunda başlatır. Bundan sonra çekirdek modu ile kullanıcı modu arasındaki geçişler işletim sistemi ve donanımın yönetiminde gerçekleşir.

Geçişin nasıl olduğunu bir örnekle görelim. Bir kullanıcı süreci çalışıyor ve bir dosya açmak, yani bir G/Ç başlatmak istiyor. `fopen` çağırıyor.

1. `fopen` bir sistem çağrısı değildir. Çalışma zamanının (runtime) birlik (unification) sağlamak ve işleri kolaylaştırmak için sunduğu üst seviye bir fonksiyondur. En alt seviyede `open` sistem çağrısını kullanır.
2. `open` çağrıldığı anda, slaytta kenara “trap” olarak yazılmış bir kesme oluşur. Mod biti 0 olur, çekirdek moduna geçilir.
3. Bu kesmenin bir işleyicisi (handler) vardır. İşleyici kesmenin bir sistem çağrısı için geldiğini ve hangi sistem çağrısı olduğunu anlar. Örneğin 300 numaralı sistem çağrısı.
4. İşleyici tabloda 300 numaralı çağrının karşılığına bakar ve gereken işleri yapar.
5. İş bitince, kullanıcı seviyesine dönmeden önce mod biti tekrar değiştirilir (1 yapılır) ve kullanıcı moduna dönülür.

Slayttaki örnekte kullanıcı modu için mod biti 1, çekirdek modu için 0 alınmıştır. Hoca bunun illa böyle olmak zorunda olmadığını tekrarladı.

> **Ek bilgi (derste anlatılmadı):** x86 işlemcilerde tek bir mod biti yerine, büyük olasılıkla daha çok “halka” (ring) adıyla bilinen dört ayrıcalık seviyesi bulunur. Pratikte işletim sistemleri çoğunlukla bunların ikisini kullanır: çekirdek için en ayrıcalıklı halka 0, kullanıcı programları için halka 3. Yani kavram olarak yine ikili mod uygulanmış olur.

5. Süreç Yönetimi

5.1 İşletim sisteminin görevleri

Hoca işletim sisteminin yerine getirmesi gereken başlıca görevleri saydı ve hepsinden kabaca bahsedeceğini, ayrıntıların yeri geldikçe konuşulacağını söyledi:

- süreç yönetimi (process management),
- bellek yönetimi (memory management),
- depolama yönetimi (storage management),
- G/Ç alt sistemi (I/O subsystem) oluşturmak.

Süreç yönetimi üç slaytla anlatılabilecek bir şey değildir, üzerinde uzun uzun konuşulacaktır.

5.2 Program ve süreç

> **Hocanın vurgusu:** Program ile süreç farklıdır. Aralarında bir ilişki vardır, ama fark da vardır.

Program, herhangi bir programlama dilinde, belli bir işletim sistemi ve mimari için çalışmak üzere ürettiğimiz kodlardan oluşan bir yazılımdır. Pasif bir varlıktır. İkincil bellekte bir alanda yazılı olarak durur.

Çalışmaya başlaması için belleğe alınması, sonra o bellek bölgesindeki komutların sırayla çalıştırılması gerekir. Programı belleğe alıp çalıştırmaya başlayabilmek için canlı bir varlığa ihtiyacımız vardır. Bu canlı varlığa **süreç** denir.

Hoca bu ilişkiyi nesne yönelimli programlamadaki (object oriented programming) sınıf (class) ile nesne (object) ilişkisine benzetti. Sınıf tanımlanmış bir yapıdır. Nesne ise o yapı kullanılarak oluşturulmuş, bellekte duran bir değişkendir.

5.3 Süreç hakkında tutulan bilgiler

Süreç yönetimi, programdan sürecin oluşturulmasını, yani belleğe kopyalanmasını ve sonra sürecin bütün yaşam döngüsü boyunca yapılacak işlerin yönetilmesini kapsar. Sürecin yaptığı her şeyin takip edilebilmesi gerekir. Bunun için veri yapılarına ihtiyaç vardır. Bu yapılarda örneğin şunlar tutulur:

- süreç hangi G/Ç aygıtlarına erişti,
- hangi kullanıcı tarafından çalıştırıldı,
- ne kadar CPU zamanı aldı,
- şu an hangi kaynaklara erişiyor ve o kaynaklar ne durumda,
- şu an hangi komutu çalıştırıyor, yani program sayacı (program counter) bilgisi.

Program sayacına neden ihtiyaç vardır? Çoklu programlama ortamında zaman paylaşımı da yapıyoruz. Süreçler arasında geçiş yapıyoruz: birini durdurup kenara alıyor, ötekine geçiyoruz. Bunu yapabilmek için kimin nerede kaldığını bilmemiz gerekir ki kaldığı yerden devam ettirebilelim. Bu bilgileri saklayan veri yapıları da işletim sistemi açısından sürecin bir parçasıdır.

Bir programdan başlatılan sürecin normalde tek bir akışı vardır. Ama süreç tek iş parçacıklı (single thread) olmak zorunda değildir. Çok iş parçacıklı (multi thread) bir yapı da olabilir. Bu durumda her iş parçacığı için kimin nerede kaldığı bilgisi ayrı tutulmalıdır. Yapı değiştikçe tutulan bilgiler de değişir ve genişler.

5.4 Süreç yönetiminin işlevleri

En basit hâliyle süreç yönetiminin işlevleri şunlardır:

- **Kullanıcı ve sistem süreçlerini oluşturmak ve ortadan kaldırmak.**
- **Süreçleri askıya almak ve sürdürmek:** Çalışan süreç kesintiye uğratılır, sonra çalışmaya devam ettirilir.
- **Eşzamanlama (synchronization) ve haberleşme mekanizmaları sunmak.**
- **Kilitlenme (deadlock) ile başa çıkmak.**

Ortadan kaldırmak yalnızca süreci bellekten silmek değildir. Bir süreç başlatıldığında belleğe programın kod ve veri kısmı kopyalanır, ama başka bilgiler de üretilir. Ayrıca programın çalışması sonucunda bir dönüş kodu (return code) oluşur: Program çalıştı, bitti, başarılı mı sonuçlandı, hata mı verdi, bir sıkıntı mı oldu? Bu bilgiyi işletim sistemi tutar. O programın dönüş bilgisini hiçbir zaman okumazsanız, işletim sisteminde bir yerde bir kayıt bırakmış olursunuz. Bu kayıt işletim sistemi çalıştığı sürece orada kalır ve kaynağı boşuna tutar. Bu konu süreçlerin ayrıntıları konuşulurken tekrar işlenecek.

> **Ek bilgi (derste anlatılmadı):** Sonlanmış ama dönüş değeri ana süreç tarafından `wait` ile okunmamış süreçlere UNIX sistemlerinde zombi süreç (zombie process) denir. Süreç tablosunda yer kaplamaya devam ederler. Ana süreç dönüş değerini okuyunca kayıt silinir.

Eşzamanlama ve haberleşme: Günümüzde bir sistemde her sürecin kendi başına çalışması gibi bir durum pek yoktur. Süreçler daha çok birbirinden servis alır, birbirine servis verir ve haberleşir. Bu yüzden farklı süreçler arasında eşzamanlama ve haberleşme mekanizmaları sunmak da süreç yönetiminin bir parçasıdır.

Kilitlenme: Hoca önce "süreçler kaynaklar için savaşır" dedi, sonra bunun ağır kaçtığını, ileride yarış durumu (race condition) konuşulacağı için "yarışır" demenin daha doğru olduğunu söyledi. Herkes işini tamamlamak için bazı kaynakları kullanmak isterse, farklı süreçlerin eriştiği kaynaklar arasında çakışma başlar. Örnek: Üç kaynağa ihtiyacınız var. 1. ve 2. kaynağı aldınız, ama 3. kaynak başkasının elinde. Onun da 1. ve 2. kaynağa ihtiyacı var ve onları siz tutuyorsunuz. Kimse elindekini bırakmıyor. Böylece ortada kilitlenip kalırsınız. Bunu çözecek ya da olmasını engelleyecek mekanizmalar da süreç yönetiminin parçasıdır.

6. Bellek Yönetimi

Bellek yönetimi daha önce kenarından köşesinden konuşulmuştu. Bir programı işlemcide çalıştırabilmek için o sürecin en azından bir parçasının bellekte olması gerekir. Benzer şekilde sürecin ihtiyaç duyduğu verinin de bellekte olması gerekir. Bu parçalardan hangisinin nerede olacağı, nasıl olacağı, ne zaman getirileceği ve ne zaman götürüleceği bellek yönetiminin işidir.

Zamanlama konusunda kısa vadeli (short term), orta vadeli (mid term) ve uzun vadeli (long term) zamanlayıcılar konuşulacak. Burada istenen şudur: Çoklu programlama yaparken bellekte birden fazla programa ait süreç tutalım. Biri G/Ç yaparken öbürü CPU'yu kullansın, o CPU'yu kullanırken öbürü G/Ç yapsın, böyle bir döngü oluşsun. Bunun için hangi programların belleğe alınacağına karar verilmelidir.

Bu karar rastgele verilirse ("sen gel, sen gel, sen gel") şu olabilir: Seçilen işlerin hepsi G/Ç yoğundur. Birini başlatırsınız, biraz CPU zamanı kullanır, sonra G/Ç başlatıp beklemeye geçer. Diğerini alırsınız, o da hemen G/Ç yapar, onu da G/Ç kuyruğuna atarsınız. Bir bakmışsınız bunlar tren gibi önce işlemci kuyruğunda, sonra G/Ç kuyruğunda bekliyor, sonra yine işlemci kuyruğuna dönüyor. Böyle bir yapının hiçbir anlamı yoktur. Hangi işin belleğe alınacağı, hangisinin geride bekleyeceği yönetilmelidir. Bellek yönetimi temelde bunu yapar.

7. Depolama Yönetimi ve Dosya Sistemi

7.1 Farklı depolama ortamları

Hoca her dönem depolama yönetimine kadar gelinemediğini, gelinse bile dosya sisteminin ayrıntılarında kalındığını, bu dönem oralara gelinmesini umduğunu söyledi.

Depolama yönetiminde sorun şudur: Elimizde birbirinden farklı ortamlar vardır.

Ortam	Erişim biçimi	Kapasite	Erişim hızı
Manyetik bant (teyp)	Sıralı (sequential)	Çok büyük	Oldukça düşük
Disk	Sıralı ile rastgele (random) arası	Banttan küçük, bellekten büyük	Banttan hızlı

Diskin erişim biçimi, diskin yapısına ve özelliğine bağlı olarak değişir. Hoca bunu "sıralı ile rastgele arası" diye tanımladı.

Kullanılan teknolojiler de farklıdır. Bantta, eski kaset çalarlardaki gibi çift halka değil, genelde tek göbekli büyük makaralar vardır. Fiziksel bir dişli ve ona giren bir sürücü kafası vardır. Kafa, bandı çevirerek erişilmesi gereken yere gider ve oradan okumaya başlar. Sabit diskte ise tek ya da birkaç silindir, bu silindirlere ulaşan kafalar ve bu kafaların içinde okuma noktaları vardır. Kısacası her ortamın farklı özellikleri vardır. Aygıt denetleyicisine (device controller) bile farklı biçimlerde erişmek ve farklı işlemler yapmak gerekir.

7.2 İki seviye soyutlama

Bu yüzden bütün bu işlerin soyutlanması (abstraction) gerekir. İşletim sistemi teknik olarak bunu yapar. Bunun üstüne bir seviye daha soyutlama gerekir: **dosya sistemi** (file system). Çünkü dosyalarımız ve klasörlerimiz vardır ve bunları düzenleriz. Ham veriyi (raw data) çoğu

zaman doğrudan kullanmayız. Bir bütünlük içeren, bir arada durması gereken bilgiyi bir dosyada saklarız. Bu bazen bir metin dosyası, bazen bir görüntü dosyası, bazen ikili (binary) bir veri, bazen de kodlayarak sakladığımız bir metin dosyasıdır.

Bu üst seviye soyutlama iki nedenle gereklidir:

- Herkese dosya oluşturma, silme, ekleme, çıkarma gibi özellikleri **tek tip (uniform)** biçimde sunabilmek.
- Her kullanıcının bilgilerini **güvenceye almak**. Bir başkası benim dosyalarıma erişip içeriklerini değiştirememeli, ben de başkasının dosyalarını doğrudan değiştirememeliyim.

Depolama tarafında bir ayrım daha vardır: Uzun süre saklanması gereken bilgilerimiz bir yanda, arşivlerimiz (archive) öbür yandadır. Arşiv, çok uzun süre saklanacak, bir kenara konacak, ama erişimin çok az olacağı yapıyı ifade eder. Hoca gazetelerin çok uzun zamandır arşiv kullandığını örnek verdi.

7.3 Dosya sistemi türleri sorusu

Bu sırada yazılı olarak dosya sistemleri arasındaki farklar soruldu. Hoca bunu şu anda anlatamayacağını, anlatmaya kalkarsa dersin ilerleyemeyeceğini söyledi. Dosya sistemleri dersin ilerleyen kısmında işlenecektir. Kısa cevabı şuydu: Dosya sistemi adı üstünde dosyalarımızı saklayabilmek için vardır. Günlüklü (journaling) dosya sistemleri, ext ailesi (ext2, ext3, ext4), FAT ailesi (FAT16, FAT32, genişletilmiş sürümleri) gibi pek çok dosya sistemi vardır. Her birinin kendi artıları ve eksileri, kendine özgü nüansları vardır, ama temel amaçları aynıdır: dosyaları saklamak ve geri almak. Aralarındaki farkı i-düğüm (inode), blok, doğrudan adresleme (direct addressing) ve dolaylı adresleme (indirect addressing) konuşulmadan anlatmak yavan kalır. Hoca soruyu soran öğrenciden, dosya sistemleri konusuna gelindiğinde bunu tekrar hatırlatmasını istedi.

8. Depolama Hiyerarşisi ve Veri Tutarlılığı

8.1 Hiyerarşinin seviyeleri

Hoca depolama seviyelerini slayttaki piramit üzerinden anlattı. En üstte yazmaçlar (register) vardır, sonra önbellek (cache), sonra ana bellek (main memory), sonra katı hâl diski (solid state disk), sonra manyetik disk gelir. Hoca, döner plakalı normal sabit diskleri katı hâl diski ile manyetik disk arasına, örneğin 4 ile 5 arasına “4,5” olarak yerleştirebileceğimizi söyledi. Bir an slaytta manyetik disk ile bandın birleştirildiğini sandı, sonra bunun yanlış okuma olduğunu, bandın ayrı bir seviye olarak gösterildiğini düzeltti.

Piramidin özellikleri:

- **Yazmaçlar:** En hızlı erişilen, en küçük ve en pahalı yerlerdir. İşlemcinin üzerine yazmaç yerleştirmek maliyetlidir. Bu yüzden sayıları sınırlıdır.
- **Önbellek:** L1, L2, L3 önbellekleri. Slaytta 16 MB ile sınırlanmıştı, ama artık çok daha büyükleri vardır. Özellikle çok çekirdekli işlemcilerde çekirdek sayısı ciddi biçimde arttığı ve hepsinin paylaştığı bir önbellek de bulunduğu için 32, 64 MB gibi rakamlar konuşulur.
- **Ana bellek:** Sıradan masaüstü ve dizüstü bilgisayarlarda genelde 64 GB’ın altındadır, sunucu sistemlerinde terabaytlara kadar çıkar.
- **Katı hâl diskleri:** Bunlarda da ciddi bir kapasite artışı vardır.

Önemli olan, sistemde farklı kapasite, aktarım hızı ve erişim biçimine sahip depolama alanları olduğunun farkında olmaktır.

8.2 Seviyeler arasında veri akışı ve kim yönetir?

Bu alanlar arasında bilgi akmalıdır. Diskteki bir bilgiye ihtiyacınız varsa önce ana belleğe alınır, sonra önbelleğe, sonra yazmaca gelir. Bu akışın yönetimi tek elden yapılmaz:

- **Yazmaçlar: derleyici.** Bir değerin yazmaçta olup olmayacağı, derleme sonucunda üretilen kod tarafından belirlenir. “Bunu yazmaca al, sonra şunu yap” diye komutları siz değil, derleyici sizin yerinize sıralar.

- **Önbellek: işlemci.** Burada ana belleğe erişim için kullanılan önbellek kastediliyor. İşlemci, belleğe erişim sırasında işleri hızlandırmak için o adresin önünü arkasını kopyalar. Böylece bir sonraki erişilecek bölgeyi işlemciye yakınlaştırmaya çalışır.
- **Geri kalan her şey: işletim sistemi.** Veri genelde sıcak (çok erişilen) ve soğuk (az erişilen) diye sınıflandırılır. Veriyi seviyeler arasında taşımak ve bunun nasıl yapılacağına performansa ve erişim kolaylığına göre karar vermek işletim sisteminin sorumluluğundadır.

8.3 Tutarlılık

Diskteki bir bilgiye eriştiniz, ana belleğe aldınız, ana bellekten önbelleğe, oradan da işlemcideki yazmaca geçti. Bu akış boyunca bilginin bütün kopyalarının birbiriyle tutarlı kalmasını sağlamamız gerekir. Tek işlemcili yapılarda bu yine bir sorundur, ama kolay yönetilir. Çok işlemcili sistemlerde ise oldukça karmaşık hâle gelir. Bilginin her an her yerde tutarlı olmasını sağlama sorununa **önbellek tutarlılığı (cache coherence)** denir. Bu ciddi bir sorundur ve dağıtık ortamlar (distributed environment) kısmına gelinirse orada konuşulacaktır.

9. G/Ç Alt Sistemi

İşletim sisteminden bir başka beklentimiz, giriş-çıkış işlerini yönetmesi ve donanımdaki bütün ince işleri bizden saklamasıdır. Hoca “saklamak” sözünün “aman duymasın” anlamında olmadığını, oradaki karmaşıklığı yönetmek zorunda kalmamak anlamında olduğunu söyledi.

Örnek olarak C’deki `printf` çağrısını verdi. `printf` ile ekrana çıktı alırken sizin yerinize çok sayıda işlem yapılır. `printf`’in sonuna satır sonu (newline) koymazsanız ya da özellikle `fflush` çağırmazsanız çıktı hemen yazılmaz. Çünkü çalışma zamanı kütüphanesi (runtime library) bir tampon (buffer) mekanizması sunar. Mantığı şudur: G/Ç zaten yavaş bir iştir. Siz çıktınızı gönderdiniz, ama başka çıktı da gelirse onları da biriktirip hepsini tek seferde yazmak daha iyidir.

Hoca derste daha önce bir deney önerdiklerini anlattı: Varsayılan ayarlarla, karakter karakter ekrana çıktı alan bir program ile aynı metni blok hâlinde yazan bir program arasında ciddi bir çalışma süresi farkı beklersiniz. Ama öyle olmadığını görürsünüz ve bu şaşırtıcı gelir. Nedeni arkadaki mekanizmalardır:

- Çalışma zamanı kütüphanesi tamponlar. Siz karakter karakter gönderirsiniz, bir tampon dolar, tampon dolunca içerik işletim sistemine gönderilir.
- İşletim sistemi de arkada ikinci bir tampon tutabilir.

G/Ç alt sisteminin başlıca işleri şunlardır:

- **Tamponlama (buffering):** Yukarıda anlatılan biriktirme.
- **Önbelleğe alma (caching):** Bantdan diske, diskten belleğe bilgi aktarırken bilgi arada önbelleğe alınmalıdır, çünkü ona tekrar erişilecektir. Örneğin bir dosya açtınız ve ilk baytını okudunuz. Büyük olasılıkla sonraki baytları da okuyacaksınız. Bu yüzden G/Ç başlatılırken sizin istediğiniz tek karakterlik işlem yapılmaz. Bir blok okunur, önbelleğe konur, siz tekrar eriştiğinizde cevap önbellekten verilir. Böylece iş hızlanır.
- **Tek tip aygıt sürücüsü arayüzü:** İşletim sistemi genel bir aygıt sürücüsü (device driver) arayüzü oluşturmalı ve aygıtları tek tip bir yapıya getirmelidir.

10. Koruma ve Güvenlik

10.1 Kimlik ve yetkilendirme

İşletim sisteminden bir başka beklentimiz koruma (protection) ve güvenliktir (security). Güvenlik günümüzde farklı yerlere kaymış olsa da, ikisi için de temelde şunu bilmek gerekir: Bu sistemi kimler kullanıyor? Birileri sistemi kullanıyorsa bunun kim olduğunu bilmek ve bu kişiler arasında bir yetkilendirme, bir seviyelendirme kurmak gerekir.

Hoca, yalnızca Windows kullananlara bunun garip gelebileceğini söyledi. Çünkü Windows’ta doğrudan bir kurulum yaptığınızda hemen ardından yönetici (administrator) yetkisinde, yani her şeyi yapabilen bir kullanıcı açılır. Tek kullanıcılı bir masaüstü ya da dizüstü bilgisayarda

bütün kaynaklara doğrudan erişebilen bir kullanıcı vardır ve bu yapıda on tane kullanıcı açmanın çoğu zaman bir anlamı yoktur. Bu eksik zaman zaman sonradan yapılan eklentilerle desteklenir.

Olması gereken ise şudur:

- Sistemde her kullanıcı için ayrı bir kimlik oluşturulmalıdır.
- Bu kimlikler denetlenmeli ve doğrulanmalıdır: kimlik doğrulama (authentication) ve yetkilendirme (authorization). Kimlik bilgileri (credential) doğru mu? Bu kişi gerçekten söylediği kişi mi?

Bu, güvenlik için önemlidir, çünkü güvenlik yalnız dışarıdan yapılan saldırılarla ilgili değildir. İçeriden de saldırı yapılır. Kimin kim olduğunu bilmek ve doğrulayabilmek bu yüzden önemlidir.

10.2 Yetki yükseltme ve korumalı portlar

Farklı kullanıcılar zaman zaman normal güvenlik seviyelerinin üstünde bir şey çalıştırmak zorunda kalabilir. Buna izin verilmesi gerekir. Bunu sağlayan şey **yetki yükseltmedir (privilege escalation)**: Kendi kimlik bilginizi daha üst seviyedeki bir yere çıkarırsınız. Linux'ta bu `sudo` ya da `su` ile yapılır. `su` ile başka bir kullanıcıya geçersiniz.

Buna neden ihtiyaç vardır? Sistemin korunabilmesi için normalde her kullanıcının her şeyi çalıştırma yetkisi yoktur. Koruma tam da bunun için vardır: Kaynaklara erişim yalnızca yetkili kişi tarafından yapılabilmelidir.

Hoca örneği ağ programlamadan verdi (bunun ağ programlama dersinde ayrıca görüleceğini söyledi). Port numaraları vardır. Süreçler bir portu servis olarak açar ve bekler. Örneğin bu bilgisayarda bir web servisi çalışsaydı 80 numaralı portu açıp bekleyecekti. Hoca 1024 ve altındaki port numaralarının korumalı (protected) olduğunu söyledi. Web, FTP, DNS, Telnet, SSH gibi bilinen (well-known) servislerin tamamı bu ilk 1024'lük alanda çalışır. Burada bir servis çalıştırılacaksa, bunu sistemdeki yetkili bir kullanıcı çalıştırmalıdır. Sıradan bir kullanıcının bu portlara erişmesi istenmez. Örneğin 70 numaralı portu açmak isteyen bir uygulama yazarsanız, uygulamanız doğrudan hata alır. Çalıştırabilmek için bir üst yetki seviyesine çıkmanız gerekir.

> **Ders sonrası not:** Hoca 00:54:26'da "bu bilgisayarda bir web browser olsaydı 80 portu açılacak" dedi. Portu açıp bekleyen web tarayıcısı değil, web sunucusudur (web servisi). Ayrıca hoca korumalı portlar için "1024 ve altı" dedi. Ayrıcalıklı portlar 0 ile 1023 arasındadır (toplam 1024 port, yani "ilk 1024 port" ifadesi doğrudur). 1024 numaralı port sıradan kullanıcıya açıktır.

10.3 Grup kimliği sorusu

Soru: Grup kimliği (group ID) yönetici (admin) anlamına mı geliyor, yoksa normal kullanıcı mı?

Cevap: Grup kimliğinin yöneticilikle bir ilgisi yoktur. Hoca önce bu ilişkiyi tamamen koparmak gerektiğini söyledi, sonra UNIX'teki erişim düzenini anlattı:

- Birinci seviyede kullanıcılar vardır. Her kullanıcının bir kullanıcı adı vardır ve bunların kimlikleri (ID) sayısal olarak tutulur. Örneğin birinin kimliği 1001, birininki 1005, birininki 1024 olabilir.
- 1000'in altındaki kimlikler genelde sistem servisleri ve çekirdek için ayrılmıştır (reserved).
- Kullanıcıların sistemde kendi süreçlerini çalıştırması beklenir. Ama bazen başka şeylere de erişmeleri gerekir: Örneğin bir portu açmaları, yani yetki yükseltmeleri gerekir.
- Yetki yükseltecek her kişiye doğrudan root hakkı vermek istemezsiniz. Root, yani yönetici, her şeye yetkilidir. Ona bir aygıta erişsin ya da bir port açsın diye bu yetkiyi verdiniz, ama o kişi belki gidip diskte başka bir şey sildi. Her şeye yetkili bir kullanıcı varken bunu denetleyemezsiniz.
- Bunun yerine yapı şöyle kurulur: Kullanıcılar ve gruplar vardır. Bir kullanıcıyı farklı yetki seviyesindeki bir grubun içine tanımlayabilirsiniz.

- Yalıtımı sağlamak için Linux sistemlerinde her kullanıcıya kendi kullanıcı adıyla aynı adlı bir grup açılır. Hoca kendi adını örnek verdi: “cihan” adlı kullanıcının “cihan” adlı bir grubu da olur.
- Bu kullanıcı dosyalarını web servisi üzerinden yayınlayacaksa, web servisinin grubuna da eklenir ve o grubun yetkilerine sahip olur. Böylece kullanıcının tanımladığı klasör altındaki dosyalara, web servisini yürüten süreç erişebilir ve onları servis edebilir. Tabii kullanıcının o dosyalar için tanımladığı haklar da geçerlidir.
- Dosyaya erişim de kullanıcı kimliği, grup kimliği ve diğerleri olarak düzenlenmiştir. Bir dosya üzerinde:
  - **Sahibi (user)** için okuma, yazma, çalıştırma (read, write, execute) hakları vardır.
  - Kullanıcının içinde bulunduğu **grup (group)** için okuma, yazma, çalıştırma hakları vardır.
  - Geriye sistemin geri kalanı kalır: **diğerleri (others)**. Onların erişimi de aynı biçimde tanımlanır.

  Ders sonrası not: “1000’in altındaki kimlikler sisteme ayrılır” sınırı dağıtıma göre değişir. Debian ve Ubuntu’da sıradan kullanıcılar 1000’den başlar. Büyük olasılıkla eski Red Hat sürümlerinde bu sınır 500’dü.

11. Çekirdekte Kullanılan Veri Yapıları

Çekirdek tarafında, veri yapıları dersinde görülen neredeyse bütün veri tipleri kullanılır. Hoca, veri yapıları dersinde “bunu da nerede kullanacağız” dediğiniz her şeyin bir şekilde çekirdekte kullanıldığını söyledi.

- **Bağlı liste ve çift yönlü bağlı liste (linked list, doubly linked list):** Süreç denetim bloklarını (process control block) birbirine bağlamak için kullanılır. Bağlı liste araya dinamik olarak eleman sokup çıkarabildiğiniz bir yapıdır. Süreç yönetiminde de buna ihtiyaç vardır: Oluşan bir süreç listeye eklenir, biten bir sürecin bilgisi listeden çıkarılır. İşletim sisteminin yapısı gereği listede tek yönde ya da çift yönde gitmeniz gerekebilir.
- **Dairesel liste (circular list):** Örneğin G/Ç aygıtlarının bekleme kuyruğunda (waiting FIFO) dairesel liste kullanmak oldukça mantıklıdır. Alanınız ya da boyutunuz sınırlıysa, yani içerik dinamik olsa da boyut olarak genişlemeyen bir yapı varsa dairesel yapı daha uygun olabilir.
- **İkili arama (binary search) ve ağaçlar:** Hoca bunların belli anlamlarda çok önemli olduğunu söyledi ve süreçlerin oluşumunu hatırlattı. Açılışta önyükleyici işletim sistemini depolamadan alır, belleğe kopyalar, ilklendirir ve çalıştırır. Sonra bütün sistem şu yolla yürür: İşletim sistemi kendi sürecinin bir kopyasını oluşturur. Sonra bu kopyaya depolamada duran bir programı kopyalar ve böylece o süreci değiştirir. Günümüz işletim sistemlerinde kabukta (shell) bir komut yazdığınızda da kabuk kendisinin bir kopyasını oluşturur, istediğiniz programı o kopyanın üzerine yükleyip orada çalıştırır. Programın dönüş değerini de size gösterir. Bu konu süreçler bölümünde ayrıca işlenecek.
- **Hash:** Günlük yaşamda çok kullanılır. Hocaya göre Windows kullanıcıları da sürekli hash kullanır: Sistemin çalışması için oluşturulan bazı servisler hash mantığı üzerine kurulu yapılarla verilir.

Hoca 1. bölüm slaytlarının 46’dan sonra 50 küsure kadar devam ettiğini, ama bu grupta biraz geride kalındığı için oralara girmeyeceğini söyledi. Anlatılmayan slaytların da okunmasını önerdi.

> **Hocanın vurgusu:** “Bu slaytları anlatmayacağım” demem, o slaytlardan sorumlu olmadığınız anlamına gelmez. Geri dönüp okumanızda fayda var.

12. İşletim Sistemi Yapıları: Servisler

12.1 Bölümün planı

Bu noktada 2. bölümün (işletim sistemi yapıları) slaytlarına geçildi. Bölümde özellikle sistem çağrıları üzerinde durulacaktı, çünkü bu, birçok şeyin nasıl döndüğünü anlamaya yardım eden önemli bir mekanizmadır. Hoca slaytta çok başlık olduğunu, ama sistem arayüzü kısmının üzerine “destanlar yazılacak” bir konu olmadığını söyledi.

İşletim sistemi bizim onunla etkileşime girmemizi sağlayan çeşitli yapılar sunar. Günümüzde bu çoğu zaman grafik kullanıcı arayüzü (graphical user interface, GUI) üzerinden olur. Ama arkada her zaman komutları doğrudan yazabildiğimiz bir yapı da vardır: komut satırı arayüzü (command line interface, CLI). Buna bazen konsol (console), bazen kabuk (shell) denir. Adlandırma işletim sisteminden işletim sistemine değişir.

Ardından sistem çağrıları gelir. Sistem çağrıları her şeyin temelidir: İşletim sisteminden servis almanızı sağlayan yapıdır. Sistem programları da kullanıcı programları da işletim sisteminden bu yapı üzerinden servis alır. Bölümün devamında işletim sisteminin tasarımı, yapısı ve hata ayıklama (debugging) konuşulacak ve sistem çağrılarının bu tasarımlar üzerinde ne kadar etkili olduğu görülecekti.

12.2 Servisler

İşletim sistemi, kullanıcı programlarının ve servislerin çalışabilmesi için bir ortam sunmak zorundadır. Temel servisler şunlardır:

- **Kullanıcı arayüzü:** Grafik ya da komut satırı olabilir. Toplu iş (batch) sistemlerindeki gibi, art arda bir grup işi tetikleyen daha otomatik bir yapı da olabilir. Hoca günümüzde doğrudan böyle çalışan bir sistem olmadığını, ama komut satırında komutları art arda dizerek bazı işleri otomatik olarak yaptırabildiğinizi, bunu ders uygulamasında da göreceğinizi söyledi.
- **Program çalıştırma:** Arayüz ne olursa olsun, istediğimiz uygulamayı çalıştırabilmeliyiz. Bunun için ikincil bellekteki program belleğe kopyalanmalı ve orada komut komut çalıştırılmalıdır.
- **G/Ç işlemleri:** Süreçler CPU yoğun (CPU bound) ve G/Ç yoğun (I/O bound) kısımlar arasında geçiş yapan bir döngüye sahiptir. Her geçişte yoğunluk aynı olmak zorunda değildir. Bir işlem yaparız, bir sonuç üretiriz ya da devam etmek için bilgiye ihtiyaç duyarız. G/Ç başlatırız, o tamamlanınca işleme devam ederiz. Bu döngüyü sağlayacak yapı kurulmuş olmalıdır.
- **Dosya sistemi işlemleri:** Dosya oluşturma, silme, değiştirme gibi işlemler yapılabilmeli, dosya sistemindeki bütün kayıtlar ve erişim hakları yönetilebilmelidir.
- **Haberleşme (communication):** Paylaşılan bellek (shared memory) ya da mesaj aktarma (message passing) ile olabilir.
- **Hata algılama (error detection).**
- **Kaynak ayırma (resource allocation).**
- **Muhasebe (accounting).**
- **Koruma ve güvenlik.**

12.3 Haberleşme: paylaşılan bellek ve mesaj aktarma

Paylaşılan bellek denince temelde düşünülen, fiziksel bir bellek alanının birden fazla süreç arasında paylaşılmasıdır. Aynı alana okuyabilen ve yazabilen süreçlerden bahsedilir. Normalde bunun tek bir bilgisayar sistemi içinde olması beklenir. Fakat farklı bilgisayar sistemlerini de InfiniBand gibi teknolojilerle bellekten belleğe bağlamak mümkündür. Bu durumda ortaya çıkan sistem yine tek bir sistem gibi düşünülebilir.

Böyle bir yapınız yoksa ve özellikle birbirinden bağımsız yapıları haberleştirmek istiyorsanız, bilginin iki sistem arasında gidip gelmesi gerekir. O zaman **mesaj aktarma** kullanılır.

Hoca bir yanılgıya karşı uyardı: “Paylaşılan bellek yalnız fiziksel olarak bir arada olan sistemlerde, mesaj aktarma yalnız ayrı sistemlerde kullanılır” diye düşünmeyin. Mesaj aktarma, yapısının getirdiği kolaylıklar ve avantajlar yüzünden tek bir sistem içinde de kullanılır.

12.4 Boru (pipe) sorusu

Bu sırada yazılı olarak yalnızca “pipe?” diye bir soru geldi. Hoca bunun bir soru olmadığını söyledi. “Pipe bir haberleşme yöntemi mi?”, “Pipe paylaşılan bellek mi?”, “Mesaj aktarma ile mi gerçekleştirilir?” gibi neyin sorulduğu belli değildi. Özellikle yazıyla soru sorulurken sorunun baştan sona net, temiz ve anlaşılır biçimde ifade edilmesini istedi. Süreçler arası iletişim (interprocess communication) ayrıca ayrıntılı konuşulacaktı. Ardından soru netleştirildi.

Soru: Boru, paylaşılan bellek için bir örnek midir?

Cevap: Hayır, olamaz. Boru, birbiriyle ilişkili iki süreç arasında haberleşme için vardır. İlişkiden kastedilen şudur:

- Bir süreç yeni bir süreç başlatabilir, yani kendisinin bir kopyasını oluşturabilir (`fork`). Oluşturduğu sürece başka bir program da yükleyebilir. Bu iki süreç arasındaki ilişkiye ana-çocuk (parent-child) ilişkisi denir. Bu iki süreç birbiriyle doğrudan ilişkilidir.
- Böyle süreçlerde, birinci süreçte oluşturduğunuz ve eriştiğiniz bütün kaynaklara aynı erişim hakkına sahip ikinci bir süreç yaratmış olursunuz. Örneğin bir dosya açtınız ve ona erişmek için elinizde bir tanıtıcı (handle) var. `fork`'tan sonra aynı tanıtıcı diğer süreçte de vardır. İsterse o da aynı kaynağa yazabilir. İkisi aynı anda yazar ya da okursa, ortak kaynak olduğu için ortalık birbirine girer.
- Boru, bu tür süreçler arasında haberleşmek için çekirdeğin sunduğu bir yapıdır. Doğrudan paylaşılan bellek değildir. Yani bellekteki bir alanın iki sürece de hem okuma hem yazma yönünde açık olması gibi bir yapı değildir.
- Boru tek yönlü (simplex) bir yapıdır. Hoca veri iletişimi dersinden hatırlattı: Simpleks, verinin tek yönde aktığı yapıdır. Boru adı üstünde bir borudur: Bir ucundan bırakırsınız, öbür ucundan çıkar. Çıktığı uçtan bırakma şansınız yoktur. Borunun bir ucu yazılabilir, diğer ucu okunabilir.
- Önce boru oluşturulur, sonra `fork` yapılır. Böylece bu boruya sahip iki süreç olur. Biri yazma ucunu, diğeri okuma ucunu kullanır ve aralarında tek yönlü iletişim kurulur.
- İki yönlü iletişim için iki boru oluşturulur ve ters yönlerde kullanılır: Birinin yazma ucunu bir süreç, okuma ucunu öbür süreç kullanır; ikinci boruda roller tersine döner.

  Ek bilgi (derste anlatılmadı): Derste kod gösterilmedi. Aşağıdaki küçük POSIX örneği, anlatılan sırayı (önce `pipe`, sonra `fork`, bir taraf yazar, öbürü okur) gösterir. `gcc -Wall -Wextra` ile uyarısız derlenip çalıştırılmış ve "cocuk okudu: merhaba" çıktısı alınmıştır. Sondaki `wait`, 5. bölümde anlatılan dönüş değerini okuma işidir.

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int fd[2];                          /* fd[0]: okuma ucu, fd[1]: yazma ucu */
    if (pipe(fd) == -1) return 1;       /* önce boru oluşturulur */
    pid_t pid = fork();                 /* sonra fork yapılır */
    if (pid == 0) {                     /* çocuk süreç: okur */
        char buf[32];
        close(fd[1]);
        ssize_t n = read(fd[0], buf, sizeof buf - 1);
        if (n > 0) { buf[n] = '\0'; printf("cocuk okudu: %s\n", buf); }
        close(fd[0]);
        return 0;
    }
    close(fd[0]);                       /* ana süreç: yazar */
    const char *msg = "merhaba";
    write(fd[1], msg, strlen(msg));
    close(fd[1]);
    wait(NULL);                         /* çocuğun dönüş değeri okunur */
    return 0;
}
```

Hoca soru için teşekkür etti ve soru sormayı teşvik etti.

> **Hocanın vurgusu:** Soru sorun. Soru dersi etkileşimli yapar, etkileşimli derste daha çok şey öğrenirsiniz.

12.5 Hata algılama, muhasebe, kayıt tutma ve kaynak ayırma

Hata algılama. Burada kastedilen, bir kullanıcı uygulamasının iki ile ikiyi toplayıp dört bulması gibi bir hata, ya da uygulamanın mantık seviyesinde yaptığı bir hata değildir. Dersin başında konuşulan ikili moda dönmek gerekir. İşletim sistemi farklı yetkilere sahiptir ve komut kümesindeki her şeyi çalıştırabilir. Kullanıcının böyle bir yetkisi yoktur. Kullanıcı uygulaması, işletim sisteminin yapabileceği işlerden birini yapmaya kalkarsa bir kesme oluşmalıdır. Kesme oluşunca işletim sistemi devreye girer, duruma bakar, "bu süreç yetkisiz bir komut çalıştırıyor" der ve gereğini yapar. Bu çoğu zaman uygulamanın durdurulması ve kenara alınması, ilgili hatanın verilmesidir. Belleğin yazma yetkisi olmayan bir yerine yazmaya, okuma yetkisi olmayan bir yerini okumaya çalışmak da olası hatalardır. Hoca bu listenin rahatça uzatılabileceğini söyledi.

Muhasebe (accounting). Buradaki muhasebe illa para anlamında değildir. Örneğin işletim sisteminde kim neyi ne kadar kullanıyor diye bakarsınız. Buna bakabilmek için birinin bunun kaydını tutması gerekir. Muhasebe bu işi yapar: hangi kullanıcı hangi kaynağı ne kadar, ne zaman, nasıl kullandı.

Kayıt tutma. Buna benzer bir iş de koruma ve güvenlik altında yapılır: kayıt (log) tutmak. Bu her şeyin kaydı olabilir: sistem açılırken servislerin verdiği bilgiler, kullanıcının oturum açması, başarısız ve başarılı oturum açma denemeleri, kullanıcının yaptığı hareketler. Bu kayıtları zaman zaman denetim kaydı (audit log) olarak da görürsünüz. Denetim kaydında yalnız başarısız işlemler değil, başarılı işlemler de bulunur. Amaç, kimin ne zaman ne yaptığını izleyebilmektir. Böylece bir sorun anında geri dönüp her şeyi kontrol edebilirsiniz.

Hoca güvenliğin yalnız bir şeyler patladıktan sonra geri dönüp kontrol etmek olmadığını da ekledi. Bu işin bir parçası olarak korumayı da devreye almak gerekir: Sistemdeki kaynaklara erişim denetlenmeli, erişimi yapan kişinin de hem kimliği doğrulanmalı hem yetkisi denetlenmelidir.

Kaynak ayırma. Hoca bunu ayrıca anlatmaya gerek görmedi, çünkü dersin başından, hatta önceki dersten beri her şey bunun üzerine dönüyordu. İşlemci, bellek, dosya sistemi ve G/Ç aygıtları ayrılmalı, yönetilmeli, etkin biçimde paylaşılmalı ve isteyenlere kullandırılmalıdır.

12.6 Blok şema: sistem çağrıları bir duvardır

Hoca işletim sisteminin blok şemasını slayt üzerinden anlattı:

- En altta işletim sisteminin üzerine oturduğu donanım vardır.
- Onun üstünde maviyle işaretlenmiş işletim sistemi vardır. Bu, belli açılardan bütün grubu kapsar. İçinde az önce sayılan servisler bulunur: program çalıştırma, dosya işlemleri, dosya sistemleri, haberleşme, kaynak ayırma, muhasebe.
- Bu servisler hem sistem programları hem kullanıcı programları tarafından kullanılacaktır. Tam bu noktada, servislerin hemen üstünde boydan boya çekilmiş gri bir alan vardır: **sistem çağrıları** (system calls).
- Bu katman bir Çin Seddi gibi sistemi bir duvarın arkasında bırakır. Neden? Servisin kontrollü biçimde verilmesi gerekir. Servis verilirken farklı API'ler kullanılır, ama bu API'ler de arkada sistem çağrıları üzerinden çalışır. Sistem çağrılarının altındaki kaynakların ya da servislerin hiçbirine başka bir yoldan ulaşılamaz. Bu bir bariyerdir.
- Bu bariyerin üstünde kullanıcı arayüzü vardır: grafik arayüz, toplu iş ya da komut satırı. Grafik arayüzde klavye, fare ya da parmakla yapılan işlemler de buna dahildir.

Kısacası tüm bu arayüzlerden gelen istekler sistem çağrıları üzerinden işletim sistemine aktarılır.

13. Kullanıcı Arayüzleri: Komut Satırı ve Grafik Arayüz

13.1 Komut satırı nasıl çalışır?

Hoca, öğrencilerin komut satırıyla daha az içli dışlı olmuş olabileceğini düşünerek bir sonraki slayttaki ekran görüntüsüne geçti. Görüntü macOS'tan alınmış, oldukça eski bir terminal sürümüydü. Günümüzde terminaller böyle görünmez, ama temel olarak istediğimiz şeyi sağlar:

- Bulunduğumuz dizindeki (directory) dosyaları listelemek için `ls` komutu girilir.

- Bulunduğumuz dizini öğrenmek için `pwd` yazılır. Görüntüde `Users` altındaki bir kullanıcı klasöründe olunduğu görülüyordu.
- Ağdaki başka bir sisteme ulaşıp orada olup olmadığını kontrol etmek için `ping` yazılır ve yanına IP adresi eklenir. Görüntüde 192.168 ile başlayan yerel bir adres pinglenmişti. Hoca IP adresine bakarak bunun büyük ihtimalle bir ADSL, fiber ya da başka bir yönlendirici (router) olduğunu tahmin etti.

Bunların hepsini kabuk kendi başına yapmaz. Çoğu ayrı birer programdır. Kabuğun işi, bu programları çalıştırmak için bize bir arayüz sunmaktır. Örneğin `ping` aslında diskte duran bir ağ uygulamasıdır. `ping` ve IP adresini yazıp Enter'a bastığınızda kabuk şunları yapar:

1. Komutu ayrıştırır (parse). Böyle bir komut var mı, yok mu diye bakar. Zaman zaman parametrelerini de kontrol eder, ama her zaman etmek zorunda değildir. Komutta hata varsa çalıştırmaz.
2. `fork` ile kendisinin bir kopyasını oluşturur. Bu yeni süreç, kabuğun bütün özelliklerine sahip bir kopyasıdır.
3. Bu sürece `ping` programını yükler. Sürecin kod alanı `ping` programının koduyla değiştirilir, verisi de kopyalanır.
4. Süreç baştan başlayacak şekilde çalıştırılır.
5. `ping`'in ürettiği ekran çıktıları (görüntüde "64 bytes from …" satırları) ekrana gelir. Hoca bunu, programın bütün çıktısının kabuğa yönlendirilmesi olarak anlattı. Böylece her şeyin tıkır tıkır çalıştığını ekranda görürsünüz.

Hoca bu konunun daha sonra ayrıca konuşulacağını, bir laboratuvar haftasında asistanların komut satırının ne olduğunu, nasıl çalıştığını ve komutlarını anlatacağını söyledi.

> **Ek bilgi (derste anlatılmadı):** Hocanın "kopyanın kod alanını `ping`'in koduyla değiştirmek" diye anlattığı adım, POSIX sistemlerinde `exec` ailesindeki çağrılarla (`execve`, `execvp` gibi) yapılır. Kabuk genelde `fork` sonrası çocukta `exec` çağırır, ana süreçte de `wait` ile çocuğun bitmesini ve dönüş değerini bekler.

13.2 Grafik arayüz

Komut satırına alternatif, Windows'taki gibi bir grafik arayüzdür. Hoca, pencereli yapının hep Windows ile geldiğini düşünmemek gerektiğini söyledi. Bu yapı Windows'tan önce de vardı, Windows yalnızca onu meşhur etti.

Grafik arayüz her zaman masaüstü benzetmesi (desktop metaphor) üzerinden gider. Oturduğunuz ortamda bir masa ve üzerinde aletleriniz, kalemleriniz vardır. Grafik arayüz benzer bir yapıyı taklit edecek biçimde tasarlanır. Masaüstü normalde temiz bir alandır; dosyalarınıza ve kullanmak istediğiniz uygulamalara erişebileceğiniz kısayollar ve semboller bulunur (tabii masaüstünü kayıtlarla doldurup "perşembe pazarına" çevirmediyseniz).

Birçok sistem hem grafik arayüz hem komut satırı sunar. Microsoft'un grafik arayüzünün arkasında da komut çalıştırılabilecek bir komut satırı vardır. UNIX sistemlerinde, doğrudan sunucu olarak kullanılmıyorsa, hem grafik arayüz hem komut satırı bulunur. Komut satırı zaten varsayılan olarak gelen, çok daha standart yapıdır.

13.3 Sunucularda grafik arayüz neden tercih edilmez?

1. Sunucu sistemine sürekli bağlanıp grafik iş yapacağınız bir ortam yoktur.
2. Grafik arayüz teknik olarak hataya daha açıktır. Bir anda bir dosya ya da klasör silersiniz, ama en üst seviyedeki bir dosyayı ya da klasörü siliyor olabilirsiniz, ya da silmek istemediğiniz bir şeyi yanlışlıkla silebilirsiniz. Komut satırında bu hatayı yapmak daha zordur.

Bu yüzden birçok sistem, sunucunun üzerinde grafik arayüz olsa bile, yetkili kullanıcının grafik arayüzden oturum açmasına izin vermez. Ubuntu, Fedora gibi masaüstü Linux dağıtımlarında da genelde root kullanıcısının grafik arayüze girmesini engelleyen bir kısıt vardır. Root ile grafik arayüze girerseniz bir anda her şeyi alt üst etme imkânı doğar.

13.4 Dokunmatik arayüzler

Günümüzde grafik arayüzün en çok kullanıldığı yerlerden biri tabletler ve cep telefonlarıdır. Burada giriş olarak klavye ve fare değil, farklı mekanizmalar kullanılır: parmakla yapılan hareketler, mimikler, yerine göre kaş göz hareketleri bile. Hoca bunun günümüz için neredeyse uç nokta olduğunu söyledi.

14. Sistem Çağrıları ve API

14.1 Sistem çağrısı ve API ilişkisi

Bu yapının arkasında her şeyin çalışmasını sağlayan şeylerden biri sistem çağrılarıdır. Sistem çağrıları, işletim sisteminin sunduğu servislere ulaşmamızı sağlayan yapılardır. Günümüzde çoğu işletim sistemi C ya da C++ ile yazıldığı için sistem çağrıları da bu seviyede yazılmıştır.

Kullanıcı ya da sistem programlarına sistem çağrıları doğrudan "al bunu kullan" diye verilmez. Bunun yerine bir **uygulama programlama arayüzü (application programming interface, API)** sunulur.

Örnek: C'de bir fonksiyon çağırıyorsunuz. Bu `open` olabilir ya da dosya işlemi yapıyorsanız `fopen` olabilir. İkisi de size fonksiyon gibi görünür. Ama bunlardan biri fonksiyon, diğeri sistem çağrısıdır. `open` doğrudan bir sistem çağrısıdır; `open` çağırırsanız doğrudan sistem çağrısı yapmış olursunuz. `fopen` dediğinizde ise `open`'ın beraberinde getirdiği birçok şeyden soyutlanmış olursunuz.

Bu yalnız `open` ile sınırlı değildir. Bir dosya, bir soket (socket) ya da başka bir aygıt açtınız. Bunun üzerinden okuma ve yazma yapacaksınız. En basit yazma, ekrana yazmaktır: sizin adınıza önceden açılmış bir kaynağa (ekrana) doğrudan mesaj göndermek. Bunun için `printf` kullanırız. `printf` bir fonksiyondur, sistem çağrısı değildir. Sistem çağrısı karşılığı `write`'tır.

> **Hocanın vurgusu:** `fopen` bir sistem çağrısı değil, kütüphane fonksiyonudur; altında `open` sistem çağrısını kullanır. Aynı şekilde `printf` bir fonksiyondur; sistem çağrısı karşılığı `write`'tır.

Hoca `man printf` ve `man write` ile ikisine bakmayı önerdi. Bakınca API'nin sistem çağrılarını kullanmak için ne kadar güzel bir ortam sağladığını görürsünüz:

- `printf`'te bir biçim dizgisi (format string) vardır.
- Parametre sayısı konusunda bir sınırınız yoktur. Parametreleri art arda verirsiniz ve metnin içine yerleştirilirler.
- Her türlü dönüşüm (conversion) yapılır, istediğiniz tipler arasında dönüşüm yapılır.
- Sonunda mesaj ekrana basılır.

Bunu `write` ile kendiniz yapmaya kalksanız "40 takla atmanız" gerekir. Bu yüzden sistem çağrıları genellikle bir kütüphane üzerinden kullanılır.

14.2 Farklı sistemlerde API'ler

Bu kütüphane işletim sisteminden işletim sistemine değişebilir, ama belli bir tekdüzeliği de vardır. Windows'ta Win32 API vardır. POSIX tabanlı sistemlerde, yani birçok UNIX, Linux ve macOS'ta, POSIX standardının tanımladığı yapıya uygun bir API vardır.

Sistem çağrıları üç aşağı beş yukarı bütün sistemlerde benzerdir, ama adları birbirinden çok farklı olabilir. Örneğin Windows'ta `CreateFile` diye gördüğünüz sistem çağrısını UNIX'te yalnızca `open` olarak görebilirsiniz. Hoca buna takılmamak gerektiğini söyledi: "Her yiğidin yoğurt yiyişi farklıdır." Kitapta genelde tek tip adlar kullanılır: `open`, `read`, `write` gibi.

> **Ders sonrası not:** Altyazıda "COSIX" ve "OSX tarafından, operating system exchange" diye geçen yerler POSIX'tir. POSIX, büyük olasılıkla "Portable Operating System Interface" sözünün kısaltmasıdır ve IEEE tarafından tanımlanmış bir standarttır.

14.3 Dosya kopyalama örneği

Tek tip adlar üzerinden bakarsak, bir dosyanın içeriğini başka bir dosyaya kopyalamak gibi basit bir işte bile ne kadar çok sistem çağrısı yapıldığını görebiliriz. Slayttaki şemada solda kaynak dosya, sağda hedef dosya, arada mavi kutu içinde yapılan işler vardı.

Hoca slaytta bir sıra hatası olduğunu söyledi: İlk satırda "Acquire input file name", sonra "Write prompt to screen" yazıyordu. Hocaya göre bunun tersi olmalıydı. Programlama dillerinden hatırlarsınız: Ekrana hiçbir şey yazmadan kullanıcıdan bilgi girmesini isterseniz tuhaf olur. Önce ne istediğinizi yazmalı, sonra bilgiyi istemelisiniz. Hocaya göre hem girdi hem çıktı dosyasının adı için aynı hata yapılmıştı.

Hocanın anlattığı akış şöyledir:

1. **Kaynak dosya adını al:** Ekrana bir istem (prompt) yaz. Bu bir sistem çağrısıdır. Sonra kullanıcının girdisini kabul et (accept input). Bu da okuma yapan bir sistem çağrısıdır.
2. **Hedef dosya adını al:** Aynı şey hedef için bir kez daha yapılır.
3. **Kaynak dosyayı aç (`open`):** Dosya olmayabilir. Slayttaki ifadeyle "if file does not exist, abort". Abort, anormal sonlanma (abnormal termination) demektir ve o da bir sistem çağrısıdır. Yani `open` bir sistem çağrısıdır, sonucunda oluşan duruma tepki de bir sistem çağrısıdır.
4. **Hedef dosyayı oluştur:** Dosya zaten varsa ne olur? Bu, `open` sistem çağrısına verdiğiniz parametrelere göre değiştirilebilir. Ama varsayılan bırakılırsa, dosya zaten var olduğu için, üzerine yazmayı (override) etkinleştirmediğiniz sürece üzerine yazamazsınız. Bu durumda da anormal sonlanma olur.
5. **Döngü:** İki dosya da açıldıysa kaynaktan okursunuz (bir sistem çağrısı), hedefe yazarsınız (bir sistem çağrısı). Döngüdeki her okuma ve her yazma ayrı bir sistem çağrısıdır. Bu, okuma başarısız olana kadar sürer: Okuma sistem çağrısının sonucu size dosyanın sonuna gelindiğini söyler.
6. **Kapat ve bitir:** Her şeyi kapatırsınız ve sonlandırırsınız. Normal sonlanma da bir sistem çağrısıdır.

   Ders sonrası not: Hoca slayttaki "Acquire input file name" satırının "Write prompt to screen" satırından önce gelmesini sıra hatası olarak yorumladı. Kitaptaki şemada büyük olasılıkla "Acquire input file name" bir başlıktır; altındaki girintili "Write prompt to screen" ve "Accept input" satırları bu adımı gerçekleştiren sistem çağrılarıdır. Bu durumda slayt da önce istemin yazıldığını, sonra girdinin okunduğunu söyler; yani hocanın anlattığı sırayla aynıdır. Kısa nota göre slayt karesi bunu doğrulamaktadır.

14.4 `read` API'si

Slaytta standart bir API örneği olarak `read` verilmişti. Bunu kendi sisteminizde `man` sayfasından da görebilirsiniz:

```
ssize_t read(int fd, void *buf, size_t count);
```

- `fd`: dosya tanımlayıcısı (file descriptor). Hoca dikkat çekti: Bu bir dosya işaretçisi (file pointer) değil, dosya tanımlayıcısıdır.
- `buf`: okunan bilginin konulacağı tampon alanı.
- `count`: okunacak bilginin boyutu.

Yani "bu dosya tanımlayıcısından bilgi okuyacağım; okuduğumu nereye koyacağım ve ne kadar bilgi okuyacağım" bilgilerini verirsiniz. Bu, tek tip hâle getirilmiş biçimdir. Arka tarafta çok daha karmaşık işler döner. Burada yalnız bir fonksiyon çağrısı hâline getirilmiştir; aslında bir sistem çağrısıdır, ama basitleştirilmiş, paketlenmiş, temizlenmiş ve arkada yapılması gereken işlerden soyutlanmıştır.

> **Ders sonrası not:** `read`'in `FILE *` almadığı, tam sayı bir dosya tanımlayıcısı aldığı doğrudur. `FILE *` alan karşılık kütüphane fonksiyonu `fread`'dir. Yukarıdaki imza 12.4'teki boru örneğinde de kullanılmış ve `gcc -Wall` ile uyarısız derlenmiştir.

15. Sistem Çağrısının İşlenmesi ve Parametre Aktarma

15.1 Sistem çağrısının işlenmesi

Sistem çağrısı işletim sistemine bir kesmeyle aktarılır. `read`, `write` ya da `open` çağırdığınızda bir kesme oluşur. İşletim sistemi bu kesmenin neden kaynaklandığını anlamak zorundadır. Kesme mekanizmasında bir kesme vektörü (interrupt vector) vardır. Vektörde ilgili yer bulunur ve karşılık gelen kod çalıştırılır. O kod, kesmeyi oluşturan şeyin ne olduğuna bakar. Bir sistem çağrısı olduğu için onunla ilgili işler yapılır.

Her sistem çağrısının bir numarası vardır ve çağrılar bu numaralarla düzenli biçimde bir tabloda durur. Hangi sistem çağrısı çağrıldıysa (örneğin 3, 5, 100) o tabloya gidilir ve karşılığında yapılması gereken yapılır. Sonunda bir sonuç oluşur ve bu sonuç geri döndürülür.

Bu işlerin tamamını, `read` fonksiyonunu da barındıran C çalışma zamanı kütüphanesi (runtime library) sizin için yerleştirir; temiz temiz çalışmanızı sağlar.

Slayttaki şemayla özetlersek:

1. Kullanıcı uygulaması, kendisine sunulan API üzerinden `open` sistem çağrısına ulaşmak ister. Şu anda kullanıcı modundasınız.
2. `open` çağrılınca bir kesme oluşur ve çekirdek moduna geçilir.
3. Kesmenin nedeni: Kullanıcı bir G/Ç yapacak, bir sistem çağrısı yaptı. Bir indekse (tabloya) bakılır ve `open`'a karşılık yapılması gereken işler yapılır.
4. Sonuç kullanıcıya geri döndürülür.

15.2 Parametre aktarma

Yalnızca sistem çağrısının numarası her şeyi çözmeye yetmez, çünkü parametre aktarmak gerekir. Örneğin yazma yapacaksanız şunlara ihtiyacınız vardır:

- yazacağınız yer,
- yazacağınız bilginin konduğu tamponun adresi,
- boyut bilgisi.

Yani birden fazla bilgi gerekir ve bu parametreleri bir yere koymanız lazımdır. Üç yöntem vardır:

1. **Yazmaçlar (register):** Parametreleri koyabileceğiniz en hızlı ve rahat yerlerden biridir. Fakat özellikle Intel mimarisine bakarsanız yazmaç kümesi oldukça sınırlıdır. Bunların bir kısmı zaten tamamen göreve özgüdür. Hepsini kullanmanız mümkün değildir ve hangisinin o an ne için kullanıldığını bilmeniz de mümkün değildir. Bu yüzden yazmaçlar hızlı bir yöntem gibi görünse de Intel mimarisinde çok faydalı ya da sürdürülebilir olmaktan uzaktır.
2. **Bellekte blok (tablo):** Aktarmanız gereken bütün parametreleri bir kenara, çoğu zaman belleğe koyarsınız ve oranın başlangıç adresini (bir yazmaçla) geçirirsiniz. "Parametrelerim bu alanda, oradan oku" demiş olursunuz. Hocaya göre bu yöntem hem Linux'ta hem Solaris'te kullanılır.
3. **Yığıt (stack):** Parametrelerin hepsi yığıta itilir (push). Geçiş yapıldıktan sonra karşı taraf, yani çekirdek tarafı, bunları çekerek (pop) alır.

Hoca bunları iki yöntem grubu olarak topladı: bir yanda yazmaçlar, öbür yanda tablo ya da yığıt. Tablo ve yığıt yöntemlerinde aktarabileceğiniz parametrelerin sayısı ve boyutu bakımından çok daha rahatsınız. Yazmaçlarla 5, 10, 20 parametre aktarmaya kalkarsanız, özellikle Intel mimarisinde bu neredeyse mümkün değildir. Blok ve yığıtın avantajı budur.

> **Ders sonrası not:** Linux'un blok yöntemini kullandığı bilgisi kitaptaki genel bir örnektir. Linux'un x86-64 sürümünde büyük olasılıkla sistem çağrısı numarası `rax` yazmacıyla, en fazla 6 parametre de doğrudan yazmaçlarla (`rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`) aktarılır. Yani 64 bit Intel mimarisinde yazmaç yöntemi pratikte kullanılmaktadır. Altyazıdaki "intern/internet mimarisi" ifadeleri Intel mimarisidir.

16. Sistem Çağrısı Türleri

16.1 Gruplar

Yüzlerce sistem çağrısı vardır. Hoca slayta bakmanın temel bir fikir vermeye yeteceğini söyledi. Başlıca gruplar:

- **Süreç denetimi (process control):** süreç oluşturma, sonlandırma, anormal sonlandırma (abort), öznitelik (attribute) alma ve değiştirme, bekleme, bir olay (event) için bekleme, sinyal gönderme ve alma, bellek dökümü (dump) alma, hata ayıklayıcı (debugger) desteği.
- **Dosya yönetimi:** dosya oluşturma, silme, ekleme yapma, var olan dosyayı açma, açılan dosyayı kapama, okuma, yazma, dosya özniteliklerini alma ve verme.
- **Aygıt yönetimi:** Dosyada olduğu gibi aygıtlar için de benzer çağrılar gerekir.
- **Bilgi bakımı:** zamanı ya da tarihi alma ve ayarlama, sistemin çalışması için çekirdek seviyesindeki sisteme özgü bir bilgiyi alma, koyma ve değiştirme.
- **Haberleşme:** Paylaşılan bellek modeli ya da mesaj aktarma için; mesaj gönderme ve alma, paylaşılan bellek alanı oluşturma ve oraya erişim hakkı kazanma.
- **Koruma:** kaynakları yetkilendirme, kullanıcıların kaynaklara erişimini açma ve kapama.

16.2 Windows ve UNIX karşılıkları

Hoca kitapla ilgili söylediğini tekrarladı: Sistem çağrılarının adlarına fazla takılmayın, kitapta tek tip adlar kullanılır. Windows ve UNIX'teki örnekler oldukça farklıdır:

İşlem	Windows	UNIX
Süreç oluşturma	`CreateProcess()`	`fork()`
Süreçten çıkma	`ExitProcess()`	`exit()`
Bekleme	`WaitForSingleObject()`	`wait()`

UNIX tarafında yazanlar, POSIX uyumlu bütün işletim sistemlerinde üç aşağı beş yukarı aynı işi yapar ve aynı ada sahiptir.

16.3 `printf` ve `write` şeması

Son olarak daha önce konuşulan şeyin başka bir şema hâli gösterildi. Bir `printf` kütüphane çağrısı yaparsınız. Bunu standart kütüphaneden, standart giriş-çıkıştan (`stdio`) çağırırsınız. `printf` aslında `write`'ın üst seviye hâlidir. Şemanın altında standart C kütüphanesi vardır. Bu kütüphane bir `write` sistem çağrısı yapar. O sistem çağrısının karşılığında yapılması gereken işler yapılır; `printf`'e verdiğiniz parametreler de yerine getirilir. Sonra size dönüş yapılır.

Bundan sonra işletim sistemi tasarımı konuşulacaktı, ama ders saati dolduğu için ders burada kesildi.

Örnek Sorular

Bu derste ekrandan alınmış soru yoktur. Aşağıdakiler derste sözlü olarak sorulan ya da hocanın sınıfa yönelttiği sorulardır.

1. (derste sözlü, 00:19:39) Ayrıcalıklı bir komutu çalıştıranın işletim sistemi mi, kullanıcı mı olduğunu nasıl anlarız? Yazılıma sorabilir miyiz?

> **Çözüm (derste verildi):** Yazılıma soramayız; "işletim sistemi misin?" sorusuna "evet" diyebilir. Yalnız donanımla da çözülemez. İşlemcinin desteklediği iki çalışma modu (kullanıcı modu ve çekirdek modu) ve bunu gösteren mod biti kullanılır. Ayrıcalıklı komutlar yalnız çekirdek modunda çalışır. Kullanıcı modunda böyle bir komut denenirse tuzak (trap) oluşur ve işletim sistemi devreye girer.

2. (derste sözlü, 00:44:08) Depolama hiyerarşisinde yazmaçların ve önbelleğin yönetimi işletim sistemi tarafından mı yapılır? Değilse kim yapar?

Çözüm (derste verildi): Hayır. Bir değerin yazmaçta tutulup tutulmayacağına derleyicinin ürettiği kod karar verir. Ana bellek erişimi için kullanılan önbelleği işlemci yönetir; erişilen adresin çevresini önbelleğe çeker. Geri kalan seviyeler (ana bellek, diskler, bant) arasındaki taşımayı işletim sistemi yönetir.

3. (derste sözlü, 00:48:18) Varsayılan ayarlarla ekrana karakter karakter yazan bir program ile aynı metni blok hâlinde yazan bir program arasında neden beklenen kadar büyük bir süre farkı çıkmaz?

Çözüm (derste verildi): Çünkü hem C çalışma zamanı kütüphanesi hem işletim sistemi tamponlama yapar. Karakterler önce kütüphanenin tamponunda birikir, tampon dolunca (ya da satır sonu veya `fflush` gelince) işletim sistemine tek seferde gönderilir. İşletim sistemi de ikinci bir tampon tutabilir. Bu yüzden her karakter için ayrı bir G/Ç yapılmaz.

4. (derste sözlü, 00:39:07) Dosya sistemleri (ext ailesi, FAT ailesi, günlüklü dosya sistemleri) arasındaki farklar nelerdir?

Cevap: Hoca bu soruyu dosya sistemleri bölümüne bıraktı. Ortak amaçlarının dosyaları saklamak ve geri almak olduğunu, farkların i-düğüm, blok, doğrudan ve dolaylı adresleme konuşulmadan anlatılamayacağını söyledi.

5. (derste sözlü, 00:56:11) Grup kimliği (group ID) yönetici (admin) anlamına mı gelir, yoksa normal kullanıcı mı?

Çözüm (derste verildi): Grup kimliğinin yöneticilikle ilgisi yoktur. UNIX'te her kullanıcının sayısal bir kullanıcı kimliği vardır; kullanıcılar gruplara üye olabilir. Bir kullanıcıya belirli bir yetki (örneğin web servisi üzerinden dosya yayınlama) vermek için ona root hakkı vermek yerine, o yetkiye sahip gruba eklenir. Dosya hakları da sahip (user), grup (group) ve diğerleri (others) için ayrı ayrı okuma, yazma, çalıştırma olarak tanımlanır.

6. (derste sözlü, 01:13:09) Boru (pipe), paylaşılan bellek için bir örnek midir?

Çözüm (derste verildi): Hayır. Boru, ana-çocuk gibi ilişkili süreçler arasında haberleşme için çekirdeğin sunduğu tek yönlü (simplex) bir yapıdır. Bir ucu yazmaya, öbür ucu okumaya açıktır. Bellekteki bir alanın iki sürece birden hem okuma hem yazma için açılması değildir. Önce boru oluşturulur, sonra `fork` yapılır. İki yönlü iletişim için iki boru ters yönlerde kullanılır.

Dersin Toparlaması

Bu derste 1. bölümün son konuları tamamlanıp 2. bölüme geçildi. Önce çoklu programlamanın CPU'nun G/Ç beklerken boş kalmasını önlediği, zaman paylaşımının da işler arasında belli aralıklarla geçerek tepki süresini düşürdüğü hatırlatıldı. Süreç sayısı arttıkça takas ve sanal belleğin neden gerektiği konuşuldu. Ardından modern işletim sisteminin kesmelerle sürüldüğü, donanım ve yazılım kesmeleri, ve bir komutu kimin çalıştırdığının yalnız donanım ile yazılımın birlikte desteklediği ikili mod sayesinde anlaşılabildiği anlatıldı. Bir `fopen` çağrısının `open` sistem çağrısına, oradan tuzağa, çekirdek moduna ve sistem çağrısı tablosuna nasıl ulaştığı izlendi.

İşletim sisteminin görevleri olarak süreç, bellek ve depolama yönetimi ile G/Ç alt sistemi, sonra koruma ve güvenlik (kimlik, yetki yükseltme, kullanıcı-grup-diğerleri hakları) ele alındı. 2. bölümde işletim sistemi servisleri, sistem çağrılarının bir duvar gibi her isteğin geçtiği katman olması, kabuğun `fork` ve program yükleme ile komut çalıştırması, API ile sistem çağrısı arasındaki fark (`fopen`/`open`, `printf`/`write`), dosya kopyalamadaki sistem çağrıları, parametre aktarma yöntemleri ve sistem çağrısı türleri işlendi. Bir sonraki derste işletim sistemlerinin tasarımı ve yapısı konuşulacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.